UNITY 6000.0.62F1 · COMPLETE OFFLINE DOCUMENTATION

Royal Gambit Developer Guide

Install. Edit. Reskin. Build your release.

szekipapa77@gmail.com



CHAPTER 01

Start here: Royal Gambit

Royal Gambit is a standalone editable Unity game template. Tap your chess piece and then a highlighted destination. Play legal chess against local computer opponents, including check, checkmate, castling, promotion and en passant. Each ladder opponent can change strength, personality, colour or odds. Royal Riddles are separate mate-in-N puzzles; they do not represent online matches.

Includes 25 curated opponents, source-backed generation, a game-specific visual editor and validation, 4 guided helpers, local saving and portrait/landscape scenes. Keep the required private game Kit and publisher-owned Szekipapa77Ads layer. Provider SDK binaries are excluded. There is no bundled HLK, purchases, multiplayer server or remote LiveOps backend. Additional included libraries: 20 puzzles. Import only one game Kit into a project because the Kit classes share names.

CHAPTER 02

Requirements, installation and first play

Use Unity **6000.0.62f1**, URP **17.0.4** and Unity UI **2.0.0**. Install matching Android SDK/NDK/OpenJDK and WebGL modules in Unity Hub. Retain the supplied manifest and ProjectSettings.

1. Extract the standalone project to a short writable local path, then open it in the matching editor. Wait for package import and compilation.
2. For native unitypackage import, create a matching URP project and use Assets > Import Package > Custom Package. Import every game, Kit, Szekipapa77Ads and WebGL-template entry.
3. Run Szekipapa77 > Royal Gambit > Setup. It configures rendering and player settings; review its changes when integrating with existing work.
4. Active Input Handling must support the classic Input Manager: choose Old or Both. New Input System only is insufficient.
5. Open `Assets/Games/ChessRoyale/Scenes/ChessRoyale_Mobile.unity` and press Play. Follow the level card and first-level instructions.
6. Stop Play mode before editing level assets. Keep all .meta GUIDs. The game uses classic Unity UI Text; do not assume an extra TextMesh Pro import is required.

CHAPTER 03

Project folders and exact source map

All game-relative paths below start at `Assets/Games/ChessRoyale`. Runtime UI is built in code; editing an unrelated static Canvas does not reskin it. Scripts contains game rules and views; Editor contains authoring tools; Kit supplies UI/audio/save/platform support; Resources/chessroyale contains level JSON, art, music and sound; Settings owns URP assets; Scenes contains both entry scenes.

- `Scripts/ChessRoyaleGame.cs`
- `Scripts/ChessRoyaleGame.Debug.cs`
- `Scripts/ChessRoyaleGame.Helpers.cs`
- `Scripts/ChessRoyaleGame.Riddles.cs`
- `Scripts/ChessRoyaleGame.Screens.cs`
- `Scripts/CrAi.cs`
- `Scripts/CrArt.cs`
- `Scripts/CrBoard.cs`
- `Scripts/CrData.cs`
- `Scripts/CrEval.cs`
- `Scripts/CrGenerator.cs`
- `Scripts/CrLevels.cs`
- `Scripts/CrMate.cs`
- `Scripts/CrSearch.cs`

Main flow: ChessRoyaleGame.cs. UI and popup consumers: ChessRoyaleGame.Screens.cs. Model/generator/loading: CrBoard.cs, CrData.cs, CrGenerator.cs, CrLevels.cs. Authoring: Editor/ChessRoyaleLevelEditor.cs. Shared ad settings live outside the game at Assets/Szekipapa77Ads/Resources/AdsSettings.asset.

CHAPTER 04

Gameplay, controls and progression

Tap your chess piece and then a highlighted destination. Play legal chess against local computer opponents, including check, checkmate, castling, promotion and en passant. Each ladder opponent can change strength, personality, colour or odds. Royal Riddles are separate mate-in-N puzzles; they do not represent online matches.

The initial bank has 25 curated opponents. Read the supplied loader/generator before changing continuation rules. Play the guided opening, ordinary progress, completion and retry paths; preserve saved indexes during a visual reskin. The Mobile scene is portrait and the WebGL-named scene is landscape, but the supplied Build menu selects portrait for both build targets. Select landscape explicitly in your Build Profile if desired. Test a near-startup scene replacement as well as a normal restart.

CHAPTER 05

Tutorials and every helper

Fresh local progress triggers the first-level guidance. Newly unlocked helpers have introduction and guided-use flows. Reset only a test save when reviewing tutorials. A free guided first use can differ from ordinary charge consumption. Exact names/unlocks/free charges: Scripts/CrData.cs; effect/help strings: Scripts/CrData.cs.

Helper	Unlock	Free charges	Action and effect
Lantern	3	3	Lights up the best move with a golden arrow.
Hourglass	6	3	Takes back your last move and the reply.
Watchtower	10	3	For the rest of the game, marks your pieces in danger and the squares the enemy controls.
Oracle	14	3	For the rest of the game, rates every move you make and shows who is winning.

Activate the tool and perform its required target action: arming a tool alone does not demonstrate its effect. A successful use requires the stated preconditions from this game: show both activation and its actual effect. Exhausted charges may offer a rewarded ad; verify success/cancel/failure and grant a reward once only.

CHAPTER 06

Visual level editor: worked editing session

Open **Szekipapa77 > Royal Gambit > Level Editor**. Exact implementation: `Editor/ChessRoyaleLevelEditor.cs`.

Open the Royal Gambit editor and select an early opponent. Use Dup or Duplicate before editing. Change its visible name and one rule or board element with the exact controls below. Save explicitly and run the current entry through the editor validation and play controls. The per-game source-control table includes the current brush, geometry, target, rule and AI fields; do not treat hierarchy or resource IDs as display text.

1. Back up Resources/chessroyale/Levels/levels.json.
2. Duplicate a simple early level and edit one small element.
3. Use the game-specific validation controls and resolve every reported issue.
4. Save explicitly; stop and restart Play mode, then use Play this level or navigate to the edited index.
5. Check portrait and landscape and rebuild demo outputs after accepting data changes.

The exact current authoring controls, including tooltips and generator switches, follow. These are game-specific source references, not a generic editor feature list.

Source	Exact current code
`ChessRoyaleLevelEditor.cs:51`	`[MenuItem("Szekipapa77/Royal Gambit/Level Editor")]`
`ChessRoyaleLevelEditor.cs:54`	`[MenuItem("Szekipapa77/Royal Gambit/Validate Levels")]`
`ChessRoyaleLevelEditor.cs:120`	`if (GUILayout.Button(_dirty ? "Save *" : "Save", EditorStyles.toolbarButton, GUILayout.Width(70))) Save();`
`ChessRoyaleLevelEditor.cs:121`	`if (GUILayout.Button("Reload", EditorStyles.toolbarButton, GUILayout.Width(60)) && (!_dirty || EditorUtility.DisplayDialog("Reload", "Discard unsaved changes?", "Discard", "Cancel"))) Load();`
`ChessRoyaleLevelEditor.cs:131`	`if (GUILayout.Button("Validate", EditorStyles.toolbarButton, GUILayout.Width(65))) { if (Riddles) ValidateRiddle(_data.selPuzzle); else ValidateOne(_data.selected); }`

ChessRoyaleLevelEditor.cs:132	`if (GUILayout.Button("Validate all", EditorStyles.toolbarButton, GUILayout.Width(80))) { if (Riddles) ValidateAllRiddles(); else ValidateAll(); }`
ChessRoyaleLevelEditor.cs:135	`if (GUILayout.Button("Play this level", EditorStyles.toolbarButton, GUILayout.Width(100))) PlayThis();`
ChessRoyaleLevelEditor.cs:162	`if (GUILayout.Button("Add")) { Rec("Add opponent"); int at = Mathf.Min(_data.selected + 1, File.opponents.Count); File.opponents.Insert(at, CrGenerator.Opponent(at + 1, 0.5f, Environment.TickCount & 0xffff)); _data.selected = at; _results.Clear(); }`
ChessRoyaleLevelEditor.cs:164	`if (GUILayout.Button("Dup")) { Rec("Duplicate opponent"); var c = Cur.Clone(); c.name += " II"; c.difficulty = -1; File.opponents.Insert(_data.selected + 1, c); _data.selected++; _results.Clear(); }`
ChessRoyaleLevelEditor.cs:165	`if (GUILayout.Button("Del") && EditorUtility.DisplayDialog("Delete opponent", $"Delete step {_data.selected + 1} ({Cur.name})?", "Delete", "Cancel")) { Rec("Delete opponent"); File.opponents.RemoveAt(_data.selected); _data.selected = Mathf.Clamp(_data.selected, 0, File.opponents.Count - 1); _results.Clear(); }`
ChessRoyaleLevelEditor.cs:166	`if (GUILayout.Button("Up") && _data.selected > 0) { Rec("Move opponent"); int s = _data.selected; (File.opponents[s], File.opponents[s - 1]) = (File.opponents[s - 1], File.opponents[s]); _data.selected--; _results.Clear(); }`
ChessRoyaleLevelEditor.cs:167	`if (GUILayout.Button("Down") && _data.selected < File.opponents.Count - 1) { Rec("Move opponent"); int s = _data.selected; (File.opponents[s], File.opponents[s + 1]) = (File.opponents[s + 1], File.opponents[s]); _data.selected++; _results.Clear(); }`
ChessRoyaleLevelEditor.cs:171	`if (_results.TryGetValue(_data.selected, out var line)) EditorGUILayout.HelpBox(line, line.StartsWith("ERR", StringComparison.Ordinal) ? MessageType.Error : MessageType.Info);`

ChessRoyaleLevelEditor.cs:378	`string name = EditorGUILayout.TextField("Name", o.name);`
ChessRoyaleLevelEditor.cs:379	`string title = EditorGUILayout.TextField("Title", o.title);`
ChessRoyaleLevelEditor.cs:380	`Color accent = EditorGUILayout.ColorField("Accent", Hex(o.accent));`
ChessRoyaleLevelEditor.cs:381	`EditorGUILayout.LabelField("Intro line");`
ChessRoyaleLevelEditor.cs:383	`string win = EditorGUILayout.TextField("Says when it wins", o.winLine);`
ChessRoyaleLevelEditor.cs:384	`string lose = EditorGUILayout.TextField("Says when you win", o.loseLine);`
ChessRoyaleLevelEditor.cs:385	`int depth = EditorGUILayout.IntSlider(new GUIContent("Search depth", "Iterative deepening limit"), o.depth, 1, 7);`
ChessRoyaleLevelEditor.cs:386	`int nodes = EditorGUILayout.IntSlider(new GUIContent("Node budget", "Max positions per move"), o.nodes, 100, 200000);`
ChessRoyaleLevelEditor.cs:387	`int margin = EditorGUILayout.IntSlider(new GUIContent("Move margin (cp)", "Plays any move within this of the best (0 = always the best)"), o.margin, 0, 600);`
ChessRoyaleLevelEditor.cs:388	`float slip = EditorGUILayout.Slider(new GUIContent("Slip chance", "Chance per move of a 3x margin"), o.slip, 0, 0.5f);`
ChessRoyaleLevelEditor.cs:389	`int noise = EditorGUILayout.IntSlider(new GUIContent("Eval noise (cp)"), o.noise, 0, 120);`
ChessRoyaleLevelEditor.cs:390	`float greed = EditorGUILayout.Slider("Greed", o.greed, -1, 1);`
ChessRoyaleLevelEditor.cs:391	`float aggr = EditorGUILayout.Slider("Aggression", o.aggression, -1, 1);`
ChessRoyaleLevelEditor.cs:392	`float caut = EditorGUILayout.Slider("Caution", o.caution, -1, 1);`

ChessRoyaleLevelEditor.cs:393	`string opening = EditorGUILayout.TextField(new GUIContent("Opening (UCI)", "Favourite first moves for either colour, e.g. e2e4 e7e5"), o.opening);`
ChessRoyaleLevelEditor.cs:394	`string odds = EditorGUILayout.TextField(new GUIContent("Odds", "Pieces it starts without: q r n b p"), o.odds);`
ChessRoyaleLevelEditor.cs:395	`int resign = EditorGUILayout.IntSlider(new GUIContent("Resign when down (pawns)", "0 = never resigns"), o.resign, 0, 20);`
ChessRoyaleLevelEditor.cs:396	`float think = EditorGUILayout.Slider("Min think time (s)", o.think, 0.2f, 2.5f);`
ChessRoyaleLevelEditor.cs:397	`string note = EditorGUILayout.TextField("Designer note", o.note);`
ChessRoyaleLevelEditor.cs:407	`if (GUILayout.Button("Novice")) Preset(o, 0.05f);`
ChessRoyaleLevelEditor.cs:408	`if (GUILayout.Button("Club")) Preset(o, 0.35f);`
ChessRoyaleLevelEditor.cs:409	`if (GUILayout.Button("Expert")) Preset(o, 0.65f);`
ChessRoyaleLevelEditor.cs:410	`if (GUILayout.Button("Master")) Preset(o, 0.92f);`
ChessRoyaleLevelEditor.cs:414	`EditorGUILayout.LabelField("Predicted strength", o.Strength.ToString("0.00"));`
ChessRoyaleLevelEditor.cs:462	`_genCount = EditorGUILayout.IntSlider("Count", _genCount, 1, 40);`
ChessRoyaleLevelEditor.cs:463	`_genReplace = EditorGUILayout.Toggle("Replace range", _genReplace);`
ChessRoyaleLevelEditor.cs:464	`_genStart = EditorGUILayout.IntField(_genReplace ? "Start step (1-based)" : "Ladder number of first", _genStart);`
ChessRoyaleLevelEditor.cs:465	`_genSeed = EditorGUILayout.IntField("Seed", _genSeed);`
ChessRoyaleLevelEditor.cs:466	`_dStart = EditorGUILayout.Slider("Start difficulty", _dStart, 0, 1);`

ChessRoyaleLevelEditor.cs:467	`_dEnd = EditorGUILayout.Slider("End difficulty", _dEnd, 0, 1);`
ChessRoyaleLevelEditor.cs:468	`_dSaw = EditorGUILayout.Slider("Sawtooth climb", _dSaw, 0, 0.2f);`
ChessRoyaleLevelEditor.cs:469	`_genBreather = EditorGUILayout.IntSlider("Breather every", _genBreather, 2, 10);`
ChessRoyaleLevelEditor.cs:470	`_dDrop = EditorGUILayout.Slider("Breather drop", _dDrop, 0, 0.5f);`
ChessRoyaleLevelEditor.cs:471	`_genValidate = EditorGUILayout.Toggle("Validate generated", _genValidate);`
ChessRoyaleLevelEditor.cs:472	`if (_genValidate) _genGames = EditorGUILayout.IntSlider("Games per reference", _genGames, 1, 6);`
ChessRoyaleLevelEditor.cs:481	`if (GUILayout.Button(_genReplace ? $"Generate and replace steps {_genStart}-{_genStart + _genCount - 1}" : $"Generate and append {_genCount} opponents")) Generate();`
ChessRoyaleLevelEditor.cs:531	`if (GUILayout.Button($"<b>{i + 1}.</b> {p.name} <color=#9ab>m{p.mateIn}</color>{diff}{res}", style, GUILayout.Height(22))) { _data.selPuzzle = i; _keyMove = 0; GUI.FocusControl(null); }`
ChessRoyaleLevelEditor.cs:535	`if (GUILayout.Button("Add")) { Rec("Add riddle"); int at = Mathf.Min(_data.selPuzzle + 1, File.puzzles.Count); File.puzzles.Insert(at, new CrPuzzle { name = "New riddle", fen = "6k1/5ppp/8/8/8/8/5PPP/3R2K1 w - - 0 1", mateIn = 1, theme = "" }); _data.selPuzzle = at; _rResults.Clear(); }`
ChessRoyaleLevelEditor.cs:537	`if (GUILayout.Button("Dup")) { Rec("Duplicate riddle"); var c = CurP.Clone(); c.name += " II"; File.puzzles.Insert(_data.selPuzzle + 1, c); _data.selPuzzle++; _rResults.Clear(); }`

ChessRoyaleLevelEditor.cs:538	`if (GUILayout.Button("Del") && EditorUtility.DisplayDialog("Delete riddle", $"Delete riddle {_data.selPuzzle + 1} ({CurP.name})?", "Delete", "Cancel")) { Rec("Delete riddle"); File.puzzles.RemoveAt(_data.selPuzzle); _data.selPuzzle = Mathf.Clamp(_data.selPuzzle, 0, File.puzzles.Count - 1); _rResults.Clear(); }`
ChessRoyaleLevelEditor.cs:539	`if (GUILayout.Button("Up") && _data.selPuzzle > 0) { Rec("Move riddle"); int s = _data.selPuzzle; (File.puzzles[s], File.puzzles[s - 1]) = (File.puzzles[s - 1], File.puzzles[s]); _data.selPuzzle--; _rResults.Clear(); }`
ChessRoyaleLevelEditor.cs:540	`if (GUILayout.Button("Down") && _data.selPuzzle < File.puzzles.Count - 1) { Rec("Move riddle"); int s = _data.selPuzzle; (File.puzzles[s], File.puzzles[s + 1]) = (File.puzzles[s + 1], File.puzzles[s]); _data.selPuzzle++; _rResults.Clear(); }`
ChessRoyaleLevelEditor.cs:544	`if (_rResults.TryGetValue(_data.selPuzzle, out var line)) EditorGUILayout.HelpBox(line, line.StartsWith("ERR", StringComparison.Ordinal) ? MessageType.Error : MessageType.Info);`
ChessRoyaleLevelEditor.cs:644	`string name = EditorGUILayout.TextField("Name", p.name);`
ChessRoyaleLevelEditor.cs:645	`string theme = EditorGUILayout.TextField("Theme (shown as a hint)", p.theme);`
ChessRoyaleLevelEditor.cs:646	`string fen = EditorGUILayout.TextField("FEN", p.fen);`
ChessRoyaleLevelEditor.cs:647	`int mate = EditorGUILayout.IntSlider("Mate in", p.mateIn, 1, 4);`
ChessRoyaleLevelEditor.cs:649	`EditorGUILayout.LabelField("Validated difficulty", p.difficulty >= 0 ? p.difficulty.ToString("0.0") : "?");`
ChessRoyaleLevelEditor.cs:650	`EditorGUILayout.LabelField("Key moves", p.keys >= 0 ? p.keys.ToString() + (p.keys == 1 ? " (unique)" : "") : "?");`
ChessRoyaleLevelEditor.cs:685	`_rgCount = EditorGUILayout.IntSlider("Count", _rgCount, 1, 20);`

`ChessRoyaleLevelEditor.cs:686`	`_rgSeed = EditorGUILayout.IntField("Seed", _rgSeed);`
`ChessRoyaleLevelEditor.cs:687`	`_rgMix = EditorGUILayout.Popup("Depth", _rgMix, new[] { "All mate in 1", "Mixed 1 and 2 (curve)", "All mate in 2" });`
`ChessRoyaleLevelEditor.cs:688`	`if (GUILayout.Button($"Generate and append {_rgCount} riddles")) GenerateRiddles();`

CHAPTER 07

Generator, insertion and replacement

Use the actual generation panel and its fields listed below. Set a reproducible seed and a small count, then review the start/index and replace option before clicking Generate. The exact generation implementation is reproduced separately with line references so append, insert and replacement behavior can be distinguished. Save and reopen the data file to verify the intended entry count; runtime generated continuation is not a project-asset save.

Source	Exact current code
`ChessRoyaleLevelEditor.cs:38`	`bool _genReplace, _genValidate = true;`
`ChessRoyaleLevelEditor.cs:463`	`_genReplace = EditorGUILayout.Toggle("Replace range", _genReplace);`
`ChessRoyaleLevelEditor.cs:464`	`_genStart = EditorGUILayout.IntField(_genReplace ? "Start step (1-based)" : "Ladder number of first", _genStart);`
`ChessRoyaleLevelEditor.cs:481`	`if (GUILayout.Button(_genReplace ? $"Generate and replace steps {_genStart}-{_genStart + _genCount - 1}" : $"Generate and append {_genCount} opponents")) Generate();`
`ChessRoyaleLevelEditor.cs:493`	`int index = (_genReplace ? _genStart : File.opponents.Count + 1) + i;`
`ChessRoyaleLevelEditor.cs:506`	`if (_genReplace)`
`ChessRoyaleLevelEditor.cs:510`	`File.opponents.RemoveRange(start, remove);`
`ChessRoyaleLevelEditor.cs:511`	`File.opponents.InsertRange(start, made);`
`ChessRoyaleLevelEditor.cs:514`	`else { File.opponents.AddRange(made); _data.selected = File.opponents.Count - made.Count; }`

Worked extension: back up the bank, generate a small set at moderate difficulty using the exact per-game controls, validate them, then Save. Reopen the JSON and confirm the count changed. Do not assume every editor has the same insertion behavior: the actual implementation below distinguishes insert, append and replace. Runtime continuation does not automatically append new records to project JSON. Validation and bot scores help find problems but do not certify a pleasant human difficulty curve.

CHAPTER 08

Actual level JSON and identifiers

Canonical asset: `Assets/Games/ChessRoyale/Resources/chessroyale/Levels/levels.json`. Array order controls visible numbering. Preserve numeric/boolean types, root version fields where present and resource/save identifiers. The actual first record is shown below; use the editor for structured coordinates and block/word data.


```
{
  "name": "Pip",
  "title": "the Page",
  "avatar": 0,
  "accent": "#3E6E9A",
  "intro": "Hello! I just learned how the horsey moves. Be gentle?",
  "winLine": "Oh! Did I win? I did!",
  "loseLine": "You trapped my king! Teach me that one day.",
  "depth": 1,
  "nodes": 300,
  "margin": 420,
  "slip": 0.25,
  "noise": 60,
  "greed": 0,
  "aggression": 0,
  "caution": 0,
  "opening": "",
  "odds": "",
  "playerColor": 0,
  "resign": 0,
  "think": 0.5,
  "tip": "move",
  "unlock": "",
  "note": "curve d=0.00",
  "difficulty": 0,
  "score": 1
}
```


Root keys: opponents, puzzles. Inspect CrBoard.cs, CrData.cs, CrGenerator.cs, CrLevels.cs for field semantics and validation. Keep visible names/hints distinct from technical keys. The game-specific board, wave, card, opponent or question records are gameplay data; UI translation is separate.

CHAPTER 09

Reskin recipe: interface colours

Stop Play mode and open `Scripts/CrArt.cs`. Find `Navy`, currently `#1B2440`. Change that hex literal to `#42C9B0`, preserve its symbol/type, compile and reopen the scene. Review HUD, menus, helper cards, settings/results and both orientations. Art sprites and procedural fallback shapes can have additional colours; edit them deliberately.

Actual consumers of this symbol are listed first, then all current palette/hex assignments. Editor-only palettes do not control the runtime. Preserve state/goal contrast and rebuild after accepting a change.

Source	Exact current code
`ChessRoyaleGame.cs:219 / void BuildBoard()`	`var inner = HzUI.Panel("Inner", _boardRoot, Navy, 12); inner.raycastTarget = false;`
`ChessRoyaleGame.Screens.cs:168 / void ShowTitle()`	`tl.AddOutline(Navy, 5); tl.AddShadow(new Color(0, 0, 0, 0.5f), 8);`
`ChessRoyaleGame.Screens.cs:288 / void ShowLadder()`	`var num = HzUI.Panel("Num", row.transform, locked ? HzUI.Hex("#9AA0B0") : tour ? Plum : Navy, 26); num.raycastTarget = false;`
`CrArt.cs:69 / class/static declarations`	`if (r > 0.84f) c = Navy;`

Source	Exact current code
`ChessRoyaleGame.cs:194 / void BuildStatic()`	`var bg = HzUI.Img("Grad", _bgRoot, HzUI.Gradient(HzUI.Hex("#2B3A68"), HzUI.Hex("#151B33")), Color.white);`
`ChessRoyaleGame.cs:1046 / void RefreshMoveList()`	`sb.Append("<color=#9FB2E0>").Append(firstFull + line).Append(".</color> ");`
`ChessRoyaleGame.cs:1051 / void RefreshMoveList()`	`_moveList.text = sb.Length > 0 ? sb.ToString() : "<color=#9FB2E0>No moves yet</color>";`
`ChessRoyaleGame.Helpers.cs:17 / class/static declarations`	`static readonly Color[] HelperColor = { HzUI.Hex("#E0A526"), HzUI.Hex("#3E6E9A"), HzUI.Hex("#C8375A"), HzUI.Hex("#7B61FF") };`
`ChessRoyaleGame.Helpers.cs:45 / void BuildHelperBar()`	`var bd = HzUI.Panel("Badge", h.btn.transform, HzUI.Hex("#FF4D4D"), 24);`

`ChessRoyaleGame.Helpers.cs:52 / void BuildHelperBar()`	`var li = HzIcons.Icon(lk.transform, HzIcons.Lock, (int)(sz * 0.38f), HzUI.Hex("#D8DCEA")); li.rectTransform.Place(new Vector2(0.5f, 0.5f), new Vector2(sz, sz * 0.5f), new Vector2(0, 12));`
`ChessRoyaleGame.Helpers.cs:107 / void RefreshHelpers()`	`h.badge.GetComponent<Image>().color = ch > 0 ? HzUI.Hex("#FF4D4D") : Mint;`
`ChessRoyaleGame.Helpers.cs:306 / void ShowMoveQuality(int loss, int eval)`	`else if (loss <= 45) { word = "Good move"; c = HzUI.Hex("#7FCB5A"); snd = "oracle_good"; }`
`ChessRoyaleGame.Helpers.cs:308 / void ShowMoveQuality(int loss, int eval)`	`else if (loss <= 220) { word = "Mistake"; c = HzUI.Hex("#F08A3C"); snd = "oracle_bad"; }`
`ChessRoyaleGame.Helpers.cs:362 / void OfferCharge(string id)`	`CardButton(m, "No thanks", HzUI.Hex("#6A6F8A"), 625, 360, 80, () => CloseModal(m), 36);`
`ChessRoyaleGame.Helpers.cs:382 / void ShowUnlock(string id, Action done)`	`CardText(m, "+" + CrHelpers.FreeCharges + " free uses", 600, 600, 50, 34, HzUI.Hex("#C27A12"), true);`
`ChessRoyaleGame.Riddles.cs:213 / void ShowRiddleResult(bool solved)`	`CardButton(m, "Menu", HzUI.Hex("#7B61FF"), 750, 260, 92, () => { CloseModal(m); ShowTitle(); }, 40, 140);`
`ChessRoyaleGame.Riddles.cs:264 / void ShowRiddleList()`	`var row = HzUI.Button("Riddle" + n, content, null, locked ? HzUI.Hex("#E4DED2") : HzUI.Hex("#F6EEDD"), Color.white, () =>`
`ChessRoyaleGame.Riddles.cs:271 / void ShowRiddleList()`	`var num = HzUI.Panel("Num", row.transform, locked ? HzUI.Hex("#9AA0B0") : Plum, 26); num.raycastTarget = false;`
`ChessRoyaleGame.Riddles.cs:291 / void ShowRiddleList()`	`else g = HzIcons.Icon(row.transform, HzIcons.Star, 44, i < cr ? Gold : HzUI.Hex("#D3C8B8"));`
`ChessRoyaleGame.Screens.cs:16 / class/static declarations`	`static readonly Color CardBg = HzUI.Hex("#FFF8EA"), TextDark = HzUI.Hex("#24283D"), TextSub = HzUI.Hex("#6A6F8A");`
`ChessRoyaleGame.Screens.cs:139 / void ShowTitle()`	`var bg = HzUI.Img("BG", _title, HzUI.Gradient(HzUI.Hex("#33457A"), HzUI.Hex("#141A31")), Color.white); bg.preserveAspect = false; bg.rectTransform.Stretch();`

ChessRoyaleGame.Screens.cs:193 / void ShowTitle()	var riddles = HzUI.Button("RiddlesBtn", _title, riddlesOpen ? "Riddles" : "", riddlesOpen ? Plum : HzUI.Hex("#4B4F66"), Color.white, () =>
ChessRoyaleGame.Screens.cs:205 / void ShowTitle()	var stats = HzUI.Button("StatsBtn", _title, "Stats", HzUI.Hex("#7B61FF"), Color.white, () => ShowStats(), 42, 32);
ChessRoyaleGame.Screens.cs:208 / void ShowTitle()	var how = HzUI.Button("HowBtn", _title, "How to play", HzUI.Hex("#2F8F7A"), Color.white, () => ShowHowTo(), 40, 32);
ChessRoyaleGame.Screens.cs:226 / void ShowSettingsModal()	sound = CardButton(m, SoundLabel(), HzUI.Hex("#4C9EEB"), 190, 540, 104, null, 46);
ChessRoyaleGame.Screens.cs:228 / void ShowSettingsModal()	music = CardButton(m, MusicLabel(), HzUI.Hex("#4C9EEB"), 310, 540, 104, null, 46);
ChessRoyaleGame.Screens.cs:281 / void ShowLadder()	var row = HzUI.Button("Row" + n, content, null, locked ? HzUI.Hex("#E4DED2") : HzUI.Hex("#F6EEDD"), Color.white, () =>
ChessRoyaleGame.Screens.cs:288 / void ShowLadder()	var num = HzUI.Panel("Num", row.transform, locked ? HzUI.Hex("#9AA0B0") : tour ? Plum : Navy, 26); num.raycastTarget = false;
ChessRoyaleGame.Screens.cs:291 / void ShowLadder()	var ring = HzUI.Img("Ring", row.transform, HzUI.Circle(64), locked ? HzUI.Hex("#B8B2A6") : CrArt.Hex(o.accent, Gold)); ring.raycastTarget = false; ring.preserveAspect = false;
ChessRoyaleGame.Screens.cs:307 / void ShowLadder()	var ul = HzUI.Label("Unlock", row.transform, "Unlocks " + CrHelpers.Title(o.unlock), 24, HzUI.Hex("#C27A12"), TextAnchor.MiddleLeft, false);
ChessRoyaleGame.Screens.cs:323 / void ShowLadder()	else g = HzIcons.Icon(row.transform, HzIcons.Star, 44, i < cr ? Gold : HzUI.Hex("#D3C8B8"));
ChessRoyaleGame.Screens.cs:392 / void ShowMatchIntro(Action done)	CardText(m, CrGenerator.StyleWords(_opp), 576, 820, 40, 28, HzUI.Hex("#3E6E9A"));
ChessRoyaleGame.Screens.cs:397 / void ShowMatchIntro(Action done)	CardText(m, setup, 770, 820, 60, 28, odds != "" ? HzUI.Hex("#2F8F7A") : TextSub);
ChessRoyaleGame.Screens.cs:421 / void ShowResult(int result, string reason, int crowns, bool firstWin)	CardButton(m, "Replay", HzUI.Hex("#7B61FF"), 915, 270, 92, () => { CloseModal(m); StartMatch(_step, null, false); }, 40, 145);

`ChessRoyaleGame.Screens.cs:441 / void ShowMenu()`	`sound = CardButton(m, SoundLabel(), HzUI.Hex("#4C9EEB"), 315, 262, 96, null, 36, -139);`
`ChessRoyaleGame.Screens.cs:443 / void ShowMenu()`	`music = CardButton(m, MusicLabel(), HzUI.Hex("#4C9EEB"), 315, 262, 96, null, 36, 139);`
`ChessRoyaleGame.Screens.cs:452 / void ShowMenu()`	`CardButton(m, "Restart", HzUI.Hex("#F5A623"), 555, 540, 100, () => { CloseAllModals(); ClearMatch(); StartMatch(_step, null, false); }, 44);`
`ChessRoyaleGame.Screens.cs:457 / void ShowMenu()`	`CardButton(m, "Restart", HzUI.Hex("#F5A623"), 555, 540, 100, () => { CloseAllModals(); StartRiddle(_riddleIndex); }, 44);`
`ChessRoyaleGame.Screens.cs:459 / void ShowMenu()`	`CardButton(m, "How to play", HzUI.Hex("#2F8F7A"), 675, 540, 100, () => ShowHowTo(), 44);`
`ChessRoyaleGame.Screens.cs:460 / void ShowMenu()`	`CardButton(m, "Main menu", HzUI.Hex("#7B61FF"), 795, 540, 100, () => { CloseAllModals(); ShowTitle(); }, 44);`
`ChessRoyaleGame.Screens.cs:468 / void ShowStats()`	`var m = Modal("Stats", new Vector2(880, 1180), HzUI.Hex("#7B61FF"));`
`ChessRoyaleGame.Screens.cs:517 / void ShowPromotionPicker(int from, int to, bool dropped)`	`var b = HzUI.Button("Promo" + type, card.transform, null, i == 0 ? HzUI.Hex("#F3E2B8") : HzUI.Hex("#E9E2D2"), Color.white, () =>`
`CrArt.cs:12 / class/static declarations`	`public static readonly Color Navy = HzUI.Hex("#1B2440"), NavyLight = HzUI.Hex("#2A3760"), Gold = HzUI.Hex("#F2C14E"),`
`CrArt.cs:13 / class/static declarations`	`GoldDark = HzUI.Hex("#B8862B"), Ivory = HzUI.Hex("#F6EBD2"), Ruby = HzUI.Hex("#C8375A"), Plum = HzUI.Hex("#4A1F3D"),`
`CrArt.cs:14 / class/static declarations`	`Sapphire = HzUI.Hex("#3E6E9A"), Mint = HzUI.Hex("#3CC3A0"), Ink = HzUI.Hex("#24283D"), Sub = HzUI.Hex("#6A6F8A"),`
`CrArt.cs:15 / class/static declarations`	`LightSq = HzUI.Hex("#F1E3C3"), DarkSq = HzUI.Hex("#4F7FA8"), Cream = HzUI.Hex("#FFF8EA");`

`CrArt.cs:61 / class/static declarations`	`Color body = white ? Ivory : HzUI.Hex("#8E1F45"), shade = white ? HzUI.Hex("#D9C8A2") : Plum;`
`CrData.cs:14 / class/static declarations`	`public string accent = "#3E6E9A";`
`CrGenerator.cs:13 / class/static declarations`	`static readonly string[] Accents = { "#3E6E9A", "#C8375A", "#2F8F7A", "#8A5CC2", "#D98A2B", "#4A7FC1", "#B04A6E", "#5E8C3A", "#2E6B8F", "#9C4A2E" };`

CHAPTER 10

Reskin recipe: visible menu and HUD text

Open `Scripts/ChessRoyaleGame.cs`, find the displayed literal `"Royal Gambit"`, and replace its displayed value with `"Royal Gambit Edition"`. Keep hierarchy names and technical IDs unchanged. In HzUI.Label/Button calls the first string may be an object name; change the displayed text argument. Read the method references below to distinguish construction from later refresh assignments.

Preserve interpolation braces, invariant number formatting, ON/OFF updates and line breaks. Update both initial and refresh paths, portrait and landscape. Exact names/unlocks/free charges: Scripts/CrData.cs; effect/help strings: Scripts/CrData.cs. Level names/hints remain separate JSON content. If a title appears in art, Title property or another screen, update those visible uses deliberately without renaming GameId, namespace, resources or save keys.

Hard-coded game strings do not automatically use shared localization. To translate the interface, route every display assignment through your translation table. Dictionaries, word banks and theme pools are independent gameplay assets and need separate validation. Test help, settings, results, tutorial and helper introductions after a fresh launch.

Source	Exact current code
`ChessRoyaleGame.cs:238 / void BuildBoard()`	`var fl = HzUI.Label("File", _boardRt, ((char)('a' + file)).ToString(), (int)(S * 0.2f), CoordColor(fileSq), TextAnchor.LowerRight, false);`
`ChessRoyaleGame.cs:242 / void BuildBoard()`	`var rl = HzUI.Label("Rank", _boardRt, ((char)('1' + (rankSq >> 4))).ToString(), (int)(S * 0.2f), CoordColor(rankSq), TextAnchor.UpperLeft, false);`
`ChessRoyaleGame.cs:317 / void BuildPanels()`	`_oppName = HzUI.Label("Name", oc.transform, "", L.portrait ? 44 : 46, Gold, ta);`
`ChessRoyaleGame.cs:320 / void BuildPanels()`	`_oppTitle = HzUI.Label("Title", oc.transform, "", 28, Ivory, ta, false);`
`ChessRoyaleGame.cs:323 / void BuildPanels()`	`_oppStyle = HzUI.Label("Style", oc.transform, "", 24, new Color(0.75f, 0.82f, 1f), ta, false);`
`ChessRoyaleGame.cs:332 / void BuildPanels()`	`_speech = HzUI.Label("Text", _speechRt, "", 26, Ink, TextAnchor.MiddleCenter, false);`
`ChessRoyaleGame.cs:339 / void BuildPanels()`	`_thinking = HzUI.Label("T", _thinkPill, "", 26, Gold, TextAnchor.MiddleCenter, false);`

ChessRoyaleGame.cs:346 / void BuildPanels()	_oppMat = HzUI.Label("Mat", _oppTray, "", 28, Ivory, TextAnchor.MiddleRight, false); _oppMat.fontStyle = FontStyle.Bold;
ChessRoyaleGame.cs:355 / void BuildPanels()	_youLabel = HzUI.Label("You", yc.transform, "You", 40, Ivory, TextAnchor.MiddleLeft);
ChessRoyaleGame.cs:358 / void BuildPanels()	_status = HzUI.Label("Status", yc.transform, "", 24, new Color(0.8f, 0.85f, 1f), TextAnchor.UpperLeft, false);
ChessRoyaleGame.cs:376 / void BuildPanels()	_youMat = HzUI.Label("Mat", _youTray, "", 28, Ivory, TextAnchor.MiddleRight, false); _youMat.fontStyle = FontStyle.Bold;
ChessRoyaleGame.cs:384 / void BuildPanels()	var head = HzUI.Label("Head", ml.transform, "Moves", 34, Gold);
ChessRoyaleGame.cs:386 / void BuildPanels()	_moveList = HzUI.Label("List", ml.transform, "", 26, Ivory, TextAnchor.UpperLeft, false);
ChessRoyaleGame.cs:397 / void BuildPanels()	_oracleText = HzUI.Label("Eval", _oracleBar, "", 22, Ink, TextAnchor.MiddleCenter, false); _oracleText.fontStyle = FontStyle.Bold;
ChessRoyaleGame.cs:412 / void BuildHud()	_stepLabel = HzUI.Label("Step", chip.transform, "", 40, Ivory); _stepLabel.rectTransform.Stretch(6);
ChessRoyaleGame.cs:418 / void BuildHud()	_crownLabel = HzUI.Label("Crowns", cc.transform, "0", 40, Gold, TextAnchor.MiddleLeft); _crownLabel.rectTransform.Place(new Vector2(0, 0.5f), new Vector2(110, 60), new Vector2(84, 0), new Vector2(0, 0.5f));
ChessRoyaleGame.cs:433 / void BuildHud()	v.letter = HzUI.Label("L", v.rt, "", (int)(S * 0.42f), Ink); v.letter.rectTransform.Stretch();
ChessRoyaleGame.cs:453 / void SetPieceLook(PieceView v, int piece)	v.letter.text = art ? "" : Pc.Letter(piece & 7).ToString();
ChessRoyaleGame.cs:782 / void StartMatch(int step, CrOpponent overrideOpp = null, bool resume = true)	Toast("Welcome back! Your game is restored.", Gold, 1.6f, 0.62f);
ChessRoyaleGame.cs:809 / void ApplyOpponentCard()	_youLabel.text = _playerWhite ? "You (Ivory)" : "You (Ruby)";

ChessRoyaleGame.cs:911 / void RefreshHud()	if (_mode == Mode.Riddle) _stepLabel.text = "Riddle " + _riddleIndex + " - Mate in " + _riddleMateIn;
ChessRoyaleGame.cs:912 / void RefreshHud()	else if (_mode == Mode.Match) _stepLabel.text = CrLevels.IsGrandTour(_step) ? "Grand Tour " + (_step - CrLevels.Curated) : "Step " + _step + " of " + CrLevels.Curated;
ChessRoyaleGame.cs:913 / void RefreshHud()	else _stepLabel.text = "";
ChessRoyaleGame.cs:987 / void RefreshTrays()	_youMat.text = ""; _oppMat.text = "";
ChessRoyaleGame.cs:1006 / void RefreshTrays()	_youMat.text = diff > 0 ? "+" + (diff / 100) : "";
ChessRoyaleGame.cs:1007 / void RefreshTrays()	_oppMat.text = diff < 0 ? "+" + (-diff / 100) : "";
ChessRoyaleGame.cs:1051 / void RefreshMoveList()	_moveList.text = sb.Length > 0 ? sb.ToString() : "<color=#9FB2E0>No moves yet</color>";
ChessRoyaleGame.Debug.cs:90 / public override string DebugCommand(string cmd)	case "helper": UseHelper(p.Length > 1 ? p[1] : "hint"); return "charges=[" + string.Join(",", Prog.charges) + "] hint=" + (_hintMove != 0 ? Mv.Uci(_hintMove) : "-") + " threat=" + _threat + " oracle=" + _oracle;
ChessRoyaleGame.Debug.cs:91 / public override string DebugCommand(string cmd)	case "charges": foreach (var id in CrHelpers.Ids) if (Prog.Has(id)) Prog.AddCharges(id, 9); SaveProg(); RefreshHelpers(); return "charges=" + string.Join(",", Prog.charges);
ChessRoyaleGame.Helpers.cs:17 / class/static declarations	static readonly Color[] HelperColor = { HzUI.Hex("#E0A526"), HzUI.Hex("#3E6E9A"), HzUI.Hex("#C8375A"), HzUI.Hex("#7B61FF") };
ChessRoyaleGame.Helpers.cs:27 / void BuildHelperBar()	h.btn = HzUI.Button("Helper_" + id, _hud, null, HelperColor[i], Color.white, () => UseHelper(id), 40, (int)(sz * 0.3f));
ChessRoyaleGame.Helpers.cs:28 / void BuildHelperBar()	h.btn.name = "Helper_" + id;
ChessRoyaleGame.Helpers.cs:39 / void BuildHelperBar()	h.label = HzUI.Label("Name", h.btn.transform, CrHelpers.Short(id), 26, Color.white, TextAnchor.MiddleCenter, false);

`ChessRoyaleGame.Helpers.cs:48 / void BuildHelperBar()`	`h.badgeText = HzUI.Label("BadgeText", bd.transform, "", (int)(sz * 0.24f), Color.white); h.badgeText.rectTransform.Stretch();`
`ChessRoyaleGame.Helpers.cs:53 / void BuildHelperBar()`	`h.lockText = HzUI.Label("LockText", lk.transform, "", 24, Gold, TextAnchor.MiddleCenter, false); h.lockText.fontStyle = FontStyle.Bold;`
`ChessRoyaleGame.Helpers.cs:99 / void RefreshHelpers()`	`h.lockText.text = at > 0 ? "Step " + at : "Soon";`
`ChessRoyaleGame.Helpers.cs:106 / void RefreshHelpers()`	`h.badgeText.text = ch > 0 ? ch.ToString() : "+";`
`ChessRoyaleGame.Helpers.cs:122 / void UseHelper(string id)`	`Toast(CrHelpers.Title(id) + (at > 0 ? " unlocks at step " + at : " is locked"));`
`ChessRoyaleGame.Helpers.cs:317 / void ShowMoveQuality(int loss, int eval)`	`var t = HzUI.Label("T", badge.transform, word, 36, Color.white); t.rectTransform.Stretch(4); t.AddOutline(new Color(0, 0, 0, 0.25f), 2);`
`ChessRoyaleGame.Helpers.cs:339 / void RefreshOracleBar()`	`_oracleText.text = "";`
`ChessRoyaleGame.Helpers.cs:345 / void OfferCharge(string id)`	`var m = Modal("Out of " + CrHelpers.Title(id), new Vector2(820, 680), Sapphire);`
`ChessRoyaleGame.Helpers.cs:349 / void OfferCharge(string id)`	`CardText(m, "Watch a short ad for one more " + CrHelpers.Title(id) + ".\nEvery ladder win also refills each helper by 1.", 390, 720, 130, 32, TextDark);`
`ChessRoyaleGame.Helpers.cs:350 / void OfferCharge(string id)`	`CardButton(m, "Watch ad +1", Mint, 510, 480, 104, () =>`
`ChessRoyaleGame.Helpers.cs:358 / void OfferCharge(string id)`	`Toast("+1 " + CrHelpers.Title(id), Gold);`
`ChessRoyaleGame.Helpers.cs:362 / void OfferCharge(string id)`	`CardButton(m, "No thanks", HzUI.Hex("#6A6F8A"), 625, 360, 80, () => CloseModal(m), 36);`
`ChessRoyaleGame.Helpers.cs:380 / void ShowUnlock(string id, Action done)`	`CardText(m, "The " + CrHelpers.Title(id), 440, 760, 70, 56, TextDark, true);`
`ChessRoyaleGame.Helpers.cs:382 / void ShowUnlock(string id, Action done)`	`CardText(m, "+" + CrHelpers.FreeCharges + " free uses", 600, 600, 50, 34, HzUI.Hex("#C27A12"), true);`

`ChessRoyaleGame.Helpers.cs:383 / void ShowUnlock(string id, Action done)`	`CardButton(m, "Great!", Mint, 710, 420, 110, () => { CloseModal(m); done(); }, 52);`
`ChessRoyaleGame.Riddles.cs:65 / void SetupRiddle(CrPuzzle p)`	`CardButton(m, "Let's go", Mint, 590, 400, 110, () => { CloseModal(m); BeginRiddleTurn(); }, 52);`
`ChessRoyaleGame.Riddles.cs:78 / void ApplyRiddleCard()`	`_oppTitle.text = (_riddleSide == Pc.White ? "Ivory" : "Ruby") + " to mate in " + _riddleMateIn;`
`ChessRoyaleGame.Riddles.cs:79 / void ApplyRiddleCard()`	`_oppStyle.text = string.IsNullOrEmpty(_riddle.theme) ? "" : "Theme: " + _riddle.theme;`
`ChessRoyaleGame.Riddles.cs:80 / void ApplyRiddleCard()`	`_youLabel.text = "Hearts";`
`ChessRoyaleGame.Riddles.cs:205 / void ShowRiddleResult(bool solved)`	`CardButton(m, "Next riddle", Mint, 620, 520, 110, () => { CloseModal(m); HzPlatform.Interstitial(GameId, () => StartRiddle(_riddleIndex + 1)); });`
`ChessRoyaleGame.Riddles.cs:210 / void ShowRiddleResult(bool solved)`	`CardButton(m, "Try again", Mint, 620, 520, 110, () => { CloseModal(m); StartRiddle(_riddleIndex); });`
`ChessRoyaleGame.Riddles.cs:212 / void ShowRiddleResult(bool solved)`	`CardButton(m, "Riddles", Sapphire, 750, 260, 92, () => { CloseModal(m); ShowTitle(); ShowRiddleList(); }, 40, -140);`
`ChessRoyaleGame.Riddles.cs:213 / void ShowRiddleResult(bool solved)`	`CardButton(m, "Menu", HzUI.Hex("#7B61FF"), 750, 260, 92, () => { CloseModal(m); ShowTitle(); }, 40, 140);`
`ChessRoyaleGame.Riddles.cs:236 / void UpdateRiddle()`	`if (_genAttempts >= 40000) { _genPending = false; if (_genModal != null) CloseModal(_genModal); _genModal = null; ShowTitle(); Toast("The scribe ran out of ink. Try again!"); }`
`ChessRoyaleGame.Riddles.cs:264 / void ShowRiddleList()`	`var row = HzUI.Button("Riddle" + n, content, null, locked ? HzUI.Hex("#E4DED2") : HzUI.Hex("#F6EEDD"), Color.white, () =>`
`ChessRoyaleGame.Riddles.cs:266 / void ShowRiddleList()`	`if (locked) { HzAudio.Play("illegal", 0.4f); Toast("Solve riddle " + (rn - 1) + " first"); return; }`

ChessRoyaleGame.Riddles.cs:273 / void ShowRiddleList()	HzUI.Label("N", num.transform, n.ToString(), 38, Gold).rectTransform.Stretch();
ChessRoyaleGame.Riddles.cs:276 / void ShowRiddleList()	var nm = HzUI.Label("Name", row.transform, name, 38, locked ? TextSub : TextDark, TextAnchor.MiddleLeft);
ChessRoyaleGame.Riddles.cs:279 / void ShowRiddleList()	var sl = HzUI.Label("Sub", row.transform, sub, 26, TextSub, TextAnchor.MiddleLeft, false);
ChessRoyaleGame.Riddles.cs:299 / void ShowRiddleList()	CardButton(m, "Close", Mint, h - 75, 400, 100, () => CloseModal(m), 46);
ChessRoyaleGame.Screens.cs:33 / class/static declarations	var tl = HzUI.Label("Title", ribbon.transform, title, 62, Color.white); tl.rectTransform.Stretch(6);
ChessRoyaleGame.Screens.cs:77 / bool HasModal(string name) { foreach (var m in _modals) if (m.dim && m.dim.name == name) return true; return false; }	var t = HzUI.Label("Txt", m.card, text, size, color, align, display);
ChessRoyaleGame.Screens.cs:106 / void Crowns(ModalInfo m, int n, float y, bool animate)	else { var t = CardText(m, "", y, 120, 120, 90, on ? Gold : new Color(0, 0, 0, 0.2f), false, (i - 1) * 140); t.font = HzIcons.Font; t.text = HzIcons.Star; c = null; }
ChessRoyaleGame.Screens.cs:167 / void ShowTitle()	var tl = HzUI.Label("Name", _title, "Royal Gambit", portrait ? 130 : 120, Gold);
ChessRoyaleGame.Screens.cs:171 / void ShowTitle()	var sub = HzUI.Label("Sub", _title, "Climb the Royal Ladder", 40, Ivory, TextAnchor.MiddleCenter, false);
ChessRoyaleGame.Screens.cs:178 / void ShowTitle()	var play = HzUI.Button("PlayBtn", _title, null, Mint, Color.white, () => { CloseTitle(); StartMatch(step); }, 60, 40);
ChessRoyaleGame.Screens.cs:181 / void ShowTitle()	var pl = HzUI.Label("L", play.transform, "Play", 66, Color.white); pl.AddOutline(new Color(0, 0, 0, 0.2f), 3);
ChessRoyaleGame.Screens.cs:183 / void ShowTitle()	var ps = HzUI.Label("S", play.transform, (CrLevels.IsGrandTour(step) ? "Grand Tour " + (step - CrLevels.Curated) : "Step " + step) + ": " + opp.name + " " + opp.title, 28, new Color(1, 1, 1, 0.92f), TextAnchor.MiddleCenter, false);

ChessRoyaleGame.Screens.cs:189 / void ShowTitle()	var ladder = HzUI.Button("LadderBtn", _title, "Ladder", Sapphire, Color.white, () => ShowLadder(), 46, 32);
ChessRoyaleGame.Screens.cs:193 / void ShowTitle()	var riddles = HzUI.Button("RiddlesBtn", _title, riddlesOpen ? "Riddles" : "", riddlesOpen ? Plum : HzUI.Hex("#4B4F66"), Color.white, () =>
ChessRoyaleGame.Screens.cs:195 / void ShowTitle()	if (!riddlesOpen) { HzAudio.Play("illegal", 0.4f); Toast("Royal Riddles open at step " + CrLevels.RiddlesUnlockStep); return; }
ChessRoyaleGame.Screens.cs:203 / void ShowTitle()	var lt = HzUI.Label("T", riddles.transform, "Riddles: step " + CrLevels.RiddlesUnlockStep, 30, new Color(1, 1, 1, 0.8f)); lt.rectTransform.Place(new Vector2(0.5f, 0.5f), new Vector2(bw - 80, 60), new Vector2(26, 0));
ChessRoyaleGame.Screens.cs:205 / void ShowTitle()	var stats = HzUI.Button("StatsBtn", _title, "Stats", HzUI.Hex("#7B61FF"), Color.white, () => ShowStats(), 42, 32);
ChessRoyaleGame.Screens.cs:208 / void ShowTitle()	var how = HzUI.Button("HowBtn", _title, "How to play", HzUI.Hex("#2F8F7A"), Color.white, () => ShowHowTo(), 40, 32);
ChessRoyaleGame.Screens.cs:216 / void ShowTitle()	var ct = HzUI.Label("N", cc.transform, Prog.TotalCrowns.ToString(), 44, Gold, TextAnchor.MiddleLeft); ct.rectTransform.Place(new Vector2(0, 0.5f), new Vector2(120, 70), new Vector2(92, 0), new Vector2(0, 0.5f));
ChessRoyaleGame.Screens.cs:226 / void ShowSettingsModal()	sound = CardButton(m, SoundLabel(), HzUI.Hex("#4C9EEB"), 190, 540, 104, null, 46);
ChessRoyaleGame.Screens.cs:227 / void ShowSettingsModal()	sound.onClick.AddListener(() => { HzAudio.Muted = !HzAudio.Muted; sound.SetText(SoundLabel()); HzAudio.Play("toggle", 0.6f); });
ChessRoyaleGame.Screens.cs:228 / void ShowSettingsModal()	music = CardButton(m, MusicLabel(), HzUI.Hex("#4C9EEB"), 310, 540, 104, null, 46);
ChessRoyaleGame.Screens.cs:229 / void ShowSettingsModal()	music.onClick.AddListener(() => { HzAudio.MusicMuted = !HzAudio.MusicMuted; music.SetText(MusicLabel()); HzAudio.Play("toggle", 0.6f); });
ChessRoyaleGame.Screens.cs:230 / void ShowSettingsModal()	CardButton(m, "Close", Mint, 450, 400, 100, () => CloseModal(m), 46);

`ChessRoyaleGame.Screens.cs:233 / static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";`	`static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";`
`ChessRoyaleGame.Screens.cs:234 / static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`	`static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`
`ChessRoyaleGame.Screens.cs:246 / void ShowHowTo()`	`"<b>Helpers</b> unlock as you climb: Lantern (hint), Hourglass (undo), Watchtower (threats) and Oracle (move ratings).",`
`ChessRoyaleGame.Screens.cs:251 / void ShowHowTo()`	`CardButton(m, "Got it!", Mint, 1030, 420, 104, () => CloseModal(m), 50);`
`ChessRoyaleGame.Screens.cs:281 / void ShowLadder()`	`var row = HzUI.Button("Row" + n, content, null, locked ? HzUI.Hex("#E4DED2") : HzUI.Hex("#F6EEDD"), Color.white, () =>`
`ChessRoyaleGame.Screens.cs:283 / void ShowLadder()`	`if (locked) { HzAudio.Play("illegal", 0.4f); Toast("Beat step " + (sn - 1) + " to unlock"); return; }`
`ChessRoyaleGame.Screens.cs:290 / void ShowLadder()`	`HzUI.Label("N", num.transform, tour ? "+" + (n - CrLevels.Curated) : n.ToString(), 38, Gold).rectTransform.Stretch();`
`ChessRoyaleGame.Screens.cs:296 / void ShowLadder()`	`var nm = HzUI.Label("Name", row.transform, locked && n > Prog.unlocked + 1 ? "???" : o.name, 40, locked ? TextSub : TextDark, TextAnchor.MiddleLeft);`
`ChessRoyaleGame.Screens.cs:302 / void ShowLadder()`	`var il = HzUI.Label("Info", row.transform, locked && n > Prog.unlocked + 1 ? "" : info, 26, TextSub, TextAnchor.MiddleLeft, false);`
`ChessRoyaleGame.Screens.cs:307 / void ShowLadder()`	`var ul = HzUI.Label("Unlock", row.transform, "Unlocks " + CrHelpers.Title(o.unlock), 24, HzUI.Hex("#C27A12"), TextAnchor.MiddleLeft, false);`
`ChessRoyaleGame.Screens.cs:330 / void ShowLadder()`	`HzUI.Label("T", nw.transform, "NEXT", 24, Color.white).rectTransform.Stretch();`
`ChessRoyaleGame.Screens.cs:337 / void ShowLadder()`	`CardButton(m, "Close", Mint, h - 75, 400, 100, () => CloseModal(m), 46);`
`ChessRoyaleGame.Screens.cs:368 / void ShowTip(string key, Action done)`	`CardButton(m, "Got it", Mint, 590, 400, 110, () => { CloseModal(m); done(); }, 52);`

`ChessRoyaleGame.Screens.cs:379 / void ShowRiddlesUnlocked(Action done)`	`CardButton(m, "Great!", Mint, 650, 400, 110, () => { CloseModal(m); done(); }, 52);`
`ChessRoyaleGame.Screens.cs:398 / void ShowMatchIntro(Action done)`	`CardButton(m, "Play!", Mint, 880, 420, 112, () => { CloseModal(m); done(); }, 56);`
`ChessRoyaleGame.Screens.cs:419 / void ShowResult(int result, string reason, int crowns, bool firstWin)`	`CardButton(m, CrLevels.IsGrandTour(next) && !CrLevels.IsGrandTour(_step) ? "Grand Tour!" : "Next opponent", Mint, 790, 560, 112, () => { CloseModal(m); HzPlatform.Interstitial(GameId, () => StartMatch(next, null, false)); });`
`ChessRoyaleGame.Screens.cs:420 / void ShowResult(int result, string reason, int crowns, bool firstWin)`	`CardButton(m, "Ladder", Sapphire, 915, 270, 92, () => { CloseModal(m); ShowTitle(); ShowLadder(); }, 40, -145);`
`ChessRoyaleGame.Screens.cs:421 / void ShowResult(int result, string reason, int crowns, bool firstWin)`	`CardButton(m, "Replay", HzUI.Hex("#7B61FF"), 915, 270, 92, () => { CloseModal(m); StartMatch(_step, null, false); }, 40, 145);`
`ChessRoyaleGame.Screens.cs:422 / void ShowResult(int result, string reason, int crowns, bool firstWin)`	`if (_step == CrLevels.Curated && firstWin) HzTween.Delay(1.2f, () => Toast("You beat the King! The Grand Tour begins.", Gold, 2.4f, 0.8f));`
`ChessRoyaleGame.Screens.cs:427 / void ShowResult(int result, string reason, int crowns, bool firstWin)`	`CardButton(m, "Try again", Mint, 760, 560, 112, () => { CloseModal(m); HzPlatform.Interstitial(GameId, () => StartMatch(_step, null, false)); });`
`ChessRoyaleGame.Screens.cs:428 / void ShowResult(int result, string reason, int crowns, bool firstWin)`	`CardButton(m, "Ladder", Sapphire, 885, 560, 96, () => { CloseModal(m); ShowTitle(); ShowLadder(); }, 44);`
`ChessRoyaleGame.Screens.cs:439 / void ShowMenu()`	`CardButton(m, "Resume", Mint, 190, 540, 110, () => CloseModal(m), 52);`
`ChessRoyaleGame.Screens.cs:441 / void ShowMenu()`	`sound = CardButton(m, SoundLabel(), HzUI.Hex("#4C9EEB"), 315, 262, 96, null, 36, -139);`
`ChessRoyaleGame.Screens.cs:442 / void ShowMenu()`	`sound.onClick.AddListener(() => { HzAudio.Muted = !HzAudio.Muted; sound.SetText(SoundLabel()); HzAudio.Play("toggle", 0.6f); });`
`ChessRoyaleGame.Screens.cs:443 / void ShowMenu()`	`music = CardButton(m, MusicLabel(), HzUI.Hex("#4C9EEB"), 315, 262, 96, null, 36, 139);`

`ChessRoyaleGame.Screens.cs:444 / void ShowMenu()`	`music.onClick.AddListener(() => { HzAudio.MusicMuted = !HzAudio.MusicMuted; music.SetText(MusicLabel()); HzAudio.Play("toggle", 0.6f); });`
`ChessRoyaleGame.Screens.cs:447 / void ShowMenu()`	`CardButton(m, "Resign", Ruby, 435, 540, 100, () =>`
`ChessRoyaleGame.Screens.cs:452 / void ShowMenu()`	`CardButton(m, "Restart", HzUI.Hex("#F5A623"), 555, 540, 100, () => { CloseAllModals(); ClearMatch(); StartMatch(_step, null, false); }, 44);`
`ChessRoyaleGame.Screens.cs:456 / void ShowMenu()`	`CardButton(m, "Skip riddle", Ruby, 435, 540, 100, () => { CloseAllModals(); StartRiddle(_riddleIndex + 1); }, 44);`
`ChessRoyaleGame.Screens.cs:457 / void ShowMenu()`	`CardButton(m, "Restart", HzUI.Hex("#F5A623"), 555, 540, 100, () => { CloseAllModals(); StartRiddle(_riddleIndex); }, 44);`
`ChessRoyaleGame.Screens.cs:459 / void ShowMenu()`	`CardButton(m, "How to play", HzUI.Hex("#2F8F7A"), 675, 540, 100, () => ShowHowTo(), 44);`
`ChessRoyaleGame.Screens.cs:460 / void ShowMenu()`	`CardButton(m, "Main menu", HzUI.Hex("#7B61FF"), 795, 540, 100, () => { CloseAllModals(); ShowTitle(); }, 44);`
`ChessRoyaleGame.Screens.cs:484 / void ShowStats()`	`("Helpers used", s.helpersUsed.ToString()),`
`ChessRoyaleGame.Screens.cs:493 / void ShowStats()`	`CardButton(m, "Close", Mint, 1090, 400, 100, () => CloseModal(m), 46);`
`ChessRoyaleGame.Screens.cs:511 / void ShowPromotionPicker(int from, int to, bool dropped)`	`var t = HzUI.Label("T", card.transform, "Promote to", 40, TextDark); t.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(500, 60), new Vector2(0, -40));`
`ChessRoyaleGame.Screens.cs:517 / void ShowPromotionPicker(int from, int to, bool dropped)`	`var b = HzUI.Button("Promo" + type, card.transform, null, i == 0 ? HzUI.Hex("#F3E2B8") : HzUI.Hex("#E9E2D2"), Color.white, () =>`
`ChessRoyaleGame.Screens.cs:526 / void ShowPromotionPicker(int from, int to, bool dropped)`	`if (!CrArt.HasPieceArt) { var l = HzUI.Label("L", b.transform, Pc.Letter(type).ToString(), 60, color == Pc.White ? Ink : Ivory); l.rectTransform.Stretch(); }`

`CrArt.cs:46 / class/static declarations`	`public static Sprite Helper(string id) => Load("Icons/helper_" + id);`

CHAPTER 11

Artwork, fonts, audio and licences

Replace game images under Resources/chessroyale through Unity while retaining resource keys, .meta GUIDs, Sprite/UI import settings, alpha, pivots and slicing. Procedural graphics in the art/view sources may need separate colour changes. Replacing one PNG does not change every fallback. Keep third-party font and other licence notices with the corresponding shipped assets.

Effects are under Resources/chessroyale/Sfx. Preserve each actual load key and importer settings when replacing sounds. Supplied music: Resources/chessroyale/Music/court.ogg. Preserve that load key and verify loop/compression and the independent Music toggle. Kit UI uses supplied fonts; preserve actual licence notices and verify new glyphs.

CHAPTER 12

Local saves, reset and migration

HzSave wraps PlayerPrefs and flushes local state on focus loss/quit. WebGL persistence uses browser storage tied to the hosting origin. Clearing site data or changing origin starts a new local save; this is not cloud synchronization. Inspect HzSave/PlayerPrefs calls in the actual game source before altering progression, stars, helper charges or tutorial flags.

Verify fresh launch, a completed arena, helper use, sound options, focus loss and restart. Use a dedicated test profile for resets. Keep technical save keys stable during colour/text reskins. If changing schema or keys, implement and test a migration rather than silently discarding existing users' progress.

CHAPTER 13

Advertising integration and configuration

Asset: `Assets/Szekipapa77Ads/Resources/AdsSettings.asset`. Inspect backend, useGoogleTestIds and missingSdkFallback in the publisher-owned Ads configuration. Buyer projects include adapter/integration code and exclude external provider SDK binaries. WebGL and a missing-provider fallback simulate ads; these are not live commercial impressions.

For real Android ads install the supported provider SDK from its official source into your working project, enter your own app/placement IDs and use test ads first. Verify banner/interstitial/rewarded callbacks on supported hardware. For production use your own IDs and disable simulated missing-SDK fallback. Build tooling rejects unsafe production fallback settings. No IAP, remote backend or HLK is included in this publisher edition.

CHAPTER 14

Android build and release

Install Unity 6000.0.62f1 Android modules through Hub. Select Android, run Setup and review application ID `com.szekipapa77.chessroyale`, version, scene orientation and signing. The build command writes `Builds/Android/chessroyale.apk`. Confirm BuildResult.Succeeded and the actual output file; intermediate report totals are not the final APK size.

Use your own keystore/signing and distribution settings. A test-signed demo is not a signed store release. A provider-free build simulates ads; a provider test-demo does not prove production ad-account configuration. Test real hardware touch, safe areas, focus/audio, save/relaunch and reward callbacks. Keep passwords, keystores and private service credentials out of buyer packages.

CHAPTER 15

WebGL build and hosting

Install matching WebGL support. Run Setup and Build WebGL; output is `Builds/WebGL/chessroyale/`. Serve index.html with all loader/framework/data/WASM/StreamingAssets files over HTTP(S), not file://. Match compression, Content-Encoding and MIME types to the host. Outer page sizing and actual scene orientation are different settings.

The publisher's playable demo is hosted on HyperCodeStore. Upload the actual runtime files rather than merely changing a URL. Test a fresh anonymous load, real pointer controls, audio after interaction, saved progress/reload and phone/desktop sizing. Rebuild and rehost when code/assets change. A successful HTTP page alone is not gameplay verification.

CHAPTER 16

Troubleshooting

Compile errors: restore matching packages and the full private Kit/Ads layer; only one game Kit belongs in a project. **Pink/invisible rendering:** run Setup and inspect Graphics and every Quality URP assignment. **No input:** classic input handling must be enabled; check canvas focus and blocking modals. **Missing resources:** restore exact paths, .meta GUIDs and importer settings.

Edits do not appear: stop Play, Save in the level editor, reload the canonical bank and check current saved level. **Text reverts:** update construction and refresh assignments from the exact table. **No live ads:** provider binaries are deliberately excluded; inspect your installed provider and fallback mode. **Scene reload exception:** retain owner-lifetime and generation guards on delayed callbacks. **WebGL fails:** inspect browser Console, compression headers and missing payloads.

CHAPTER 17

Support and final release checklist

Publisher: Szekipapa77. Support: szekipapa77@gmail.com. Include game title, exact Unity version, platform, first Console error and reproduction steps. Never send passwords/tokens.

1. Clean-import the standalone project and native package.
2. Play both scenes and replace a scene near startup.
3. Review tutorial, normal completion/retry and every helper activation plus effect.
4. Validate editor changes and test reskin text/colours in both layouts.
5. Rebuild affected Android/WebGL outputs.
6. Verify complete public APK bytes, actual browser demo, documentation and video links.
7. Keep licences and exclude all internal prompts, planning/capture/build/QA tooling from buyers.